

Executive Summary

This Data Governance Health Check skill for Cortex Code enables data platform and security teams to assess Snowflake account governance posture in minutes rather than hours. The skill runs 12 metadata-driven checks across data protection, identity and access, and authentication and network security, producing a RAG-scored report with prioritized findings and remediation SQL. It was built and tested against a live Snowflake trial account using Cortex Code CLI.

Design + Rationale

Problem Statement

As AI accelerates development velocity, the bottleneck shifts from building to validation, increasing the importance of robust testing frameworks to ensure outputs are accurate, safe, and compliant at scale. Data governance is one area where this gap is acutely felt; audits are manual, slow, and require deep expertise to run correctly. A governance or security team member today has to know which metadata views to query, how to interpret results, and how to prioritize findings, all before writing a single line of remediation SQL. As a result, audits are infrequent, findings get lost in spreadsheets, and risks go unaddressed.

The governance-health-check skill automates this entirely. It runs 12 checks across three domains: data protection, identity and access, and authentication and network security. The target user is a data platform lead, security engineer, or governance team member who needs a clear, shareable picture of their account's security posture in minutes rather than hours.

Skill Architecture

After reviewing the structure of existing Cortex Code skills, I adopted the orchestrator + sub-file pattern rather than building a single monolithic file: a main `SKILL.md` orchestrator that handles environment discovery, execution sequencing, scoring, and output, plus three domain sub-files (`data_protection.md`, `identity_access.md`, `auth_network.md`) that each contain embedded SQL for their respective checks.

Matching this pattern was a deliberate choice to ensure the skill feels native to practitioners already familiar with Cortex Code.

The skill draws on eight metadata sources: SNOWFLAKE.DATA_PRIVACY.CLASSIFICATION_RESULT, seven ACCOUNT_USAGE views (POLICY_REFERENCES, TAG_REFERENCES, GRANTS_TO_ROLES, USERS, LOGIN_HISTORY, ROLES, DATABASES), and the SHOW NETWORK POLICIES command.

The 12 Checks

The skill runs 12 checks across three domains. 8 are atomic checks that each establish a discrete fact about account state. 4 are cross-reference checks that surface compound risk by combining findings across domains.

Data Protection

1. PII Classification Coverage
2. Masking & Row Access Policy Coverage
3. Tag Coverage
4. PII Classified but Unmasked
5. PII Classified but Untagged

Identity & Access

6. Privilege Distribution — Overprivileged Roles
7. Humans with Powerful Roles
8. Inactive Users & MFA Disabled
9. Inactive Users with Powerful Roles

Auth & Network

10. Failed Login Patterns
11. Network Policy Existence & Application
12. Failed Logins × No Network Policy

The four cross-reference checks are the most valuable part of the skill. An inactive user is a minor finding. An inactive user who still holds ACCOUNTADMIN is a critical one. PII classified but unmasked means classification ran, but protection did not follow. These compound findings are the ones that matter most to a governance team, and they are exactly the kind of thing that gets missed when checks are run in isolation.

RAG Scoring Design

Each check scores Red, Amber, or Green based on defined thresholds. Domain scores use worst-check-wins logic. If any check in a domain scores Red, the domain scores Red. The overall score uses the same logic across domains.

Two non-obvious design decisions are worth explaining:

1. Empty results do not always mean Green. No masking policies applied is Red, not Green: absence of configuration is a finding, not a clean bill of health. But no failed logins is Green: absence of bad events is genuinely good. The empty-state behavior was defined explicitly for every check before building, because getting it wrong produces false confidence.
2. The scoring for failed logins was updated during testing. 13 failures initially triggered Red via the 11+ threshold, but pattern analysis showed all failures were benign. The scoring now evaluates whether failures show signs of credential stuffing or brute force — the same IP targeting multiple users, failures spread across many IPs, or sustained attack signatures — versus operational noise such as expired tokens and MFA typos concentrated on a single user.

Extensibility: Compliance Profiles and Context-Aware Scoring

The 12 current checks and their scoring thresholds are intentionally separated from core skill logic, making both extensible without rearchitecting anything. The orchestrator + sub-file pattern means compliance-specific modules are natural additions. A [compliance_hipaa.md](#) or [compliance_gdpr.md](#) sub-file would add checks relevant to that framework, activated via a Phase 0 scoping question consistent with how the skill already asks about database scope.

Thresholds would also change depending on the context. For example, tag coverage at 60% might be Amber for an early-stage startup and Red for a healthcare company under HIPAA.

Key Guardrails

The skill was designed around three requirements: do not assume, do not break things, and fail gracefully.

Discovery over asking. The skill discovers the current warehouse and role dynamically rather than asking the user to specify them. The user already made these decisions when they set up their session, re-asking is redundant and adds friction. The only thing the skill asks for is database scope for PII classification, because that is a genuine choice the user needs to make.

Read-only with one bounded exception. Every check is a SELECT query or SHOW command, with no writes to Snowflake. The one exception is `SYSTEM$CLASSIFY`, which writes classification tags to tables. This is explicitly user-gated: the skill asks for confirmation before running it, warns about compute cost, and only proceeds on approval.

Graceful degradation. If a source is inaccessible, the skill does not stop. It skips the dependent checks, marks them Amber, and reports coverage as a percentage. Every skip message includes the specific grant

SQL needed to fix the access issue, not just a generic error. Missing objects get the same treatment; if no masking policies exist, the finding explains what to create and how, not just that nothing was found.

Output Design

The skill produces three outputs:

1. The **terminal summary** gives the user an immediate answer — overall score, domain findings, top priorities — without requiring them to open a file.
2. The **markdown report** is the full artifact: every check, every finding, every remediation SQL command, saved and versioned by timestamp.
3. The **PDF report** exists because a governance health check is not just for the person running it. It is a document that gets shared with security stakeholders, presented in reviews, and filed as evidence of due diligence. The companion [report-style.css](#) file ensures every run produces a consistently formatted, professional report, regardless of who runs the skill or on what machine.

One constraint worth noting: ACCOUNT_USAGE views have up to 3 hours of latency depending on the view. The skill surfaces this explicitly in the report metadata so stakeholders understand that the data they are reviewing reflects a recent but not real-time snapshot of account state.

Evaluation

Test Matrix

Note: PII classification (SYSTEM\$CLASSIFY), masking policies, and row access policies require **Enterprise Edition**. All cases were tested on a **Standard Edition** account. Checks dependent on these features (1, 2, 4, 5) consistently score Red or Amber due to feature unavailability, not misconfiguration. Where test designs reference Enterprise-only preconditions (e.g., "PII classification run on scoped database"), these were tested with modified preconditions reflecting Standard Edition constraints. Observations note the differences.

Case 1: Full Run, ACCOUNTADMIN, Baseline — Happy Path

Preconditions: ACCOUNTADMIN role active, warehouse set, all 8 sources accessible, minimal tags applied to account tables, no PII classification run, permissive network policy (ALLOW_ALL_IPS) applied at account level, at least one user with MFA enabled

User Input: Session already set to ACCOUNTADMIN and warehouse. Run **governance health check**, provide valid database name when prompted

Expected Behavior: All 12 checks execute successfully with 100% coverage; PDF generated with correct RAG badges.

- **Checks 1–2:** Red — no classification or masking policies applied
- **Check 3:** Red/Amber — tag coverage below threshold
- **Checks 4–5:** Amber — classification data unavailable
- **Checks 6–9:** Based on account state (e.g., human user on powerful roles → Amber)
- **Check 10:** Green/Amber depending on failed login patterns
- **Check 11:** Amber — permissive ALLOW_ALL_IPS network policy
- **Check 12:** Red/Amber — failed logins combined with permissive network policy

Domain Scores: Data Protection = Red; Auth & Network = Amber

Overall Score: Red

This case establishes the baseline governance posture before any remediation.

Case 2: Full Run, ACCOUNTADMIN, Post-Remediation — Happy Path

Preconditions: ACCOUNTADMIN role active, warehouse set, all 8 sources accessible, PII classification run on scoped database with full coverage, masking policies applied to all classified PII columns, governance tags applied to 80%+ of tables, restrictive network policy applied at account level limiting connections to known IPs

User Input: Session already set to ACCOUNTADMIN and warehouse. Run **governance health check**, provide valid database name when prompted

Expected Behavior: All 12 checks execute successfully with 100% coverage; PDF generated with correct RAG badges.

- **Checks 1–3:** Green — classification, masking, and tag coverage meet thresholds
- **Checks 4–5:** Green — classified PII is properly masked and tagged
- **Checks 6–10, 12:** Unchanged from Case 1 (no changes to roles, users, MFA, or login history)
- **Check 11:** Green — restrictive network policy correctly applied

Domain Scores:

- Data Protection: Red → Green
- Auth & Network: Amber → Green
- Identity & Access: remains Amber

Overall Score: Improves to Amber

This case demonstrates a fully remediated account. Data Protection and Auth and Network both improve to Green, with Identity and Access as the only remaining gap.

Case 3: Failed Logins Are Benign — Edge Case

Preconditions: ACCOUNTADMIN role active, warehouse set, all 8 sources accessible, login history contains failures from the last 30 days attributable to OAuth token expiry, MFA code typos, and second factor failures. All failures originate from the same user and IP address.

User Input: Session already set to ACCOUNTADMIN and warehouse. Run **governance health check**, provide valid database name when prompted

Expected Behavior: All 12 checks execute successfully with 100% coverage.

- **Check 10:** Amber — high volume but benign; no credential stuffing or brute-force indicators
- **Check 12:** Amber — failures present but mitigated by active network policy

- **Other checks:** Score based on actual account state

User-facing output: The report contextualizes the pattern (same user, same IP, authentication friction signals) and does not flag this as a security incident.

Case 4: Human User Holds Multiple Powerful Roles — Edge Case

Preconditions: ACCOUNTADMIN role active, warehouse set, all 8 sources accessible, at least one human user (HAS_PASSWORD = TRUE, HAS_RSA_PUBLIC_KEY = FALSE) assigned to both ACCOUNTADMIN and ORGADMIN, user has logged in within the last 90 days

User Input: Session already set to ACCOUNTADMIN and warehouse. Run **governance health check**, provide valid database name when prompted

Expected Behavior: All 12 checks execute successfully with 100% coverage.

- **Check 7:** Amber — human user holds multiple powerful roles (ACCOUNTADMIN and ORGADMIN)
- **Check 9:** Green — user is active, distinguishing role risk from inactivity
- **Other checks:** Score based on actual account state

User-facing output: The report surfaces the user by name with associated roles and recommends reviewing whether both roles are necessary.

Case 5: Invalid Database Name in Phase 0 — Edge Case

Preconditions: ACCOUNTADMIN role active, warehouse set, all 8 sources accessible

User Input: Run **governance health check**. When prompted for database name, enter a nonexistent database name such as FAKE_DB_12345

Expected Behavior:

- **Phase 0 (validation):** Database name is validated against **SHOW DATABASES** and determined to be invalid
- **Execution:** Skill does not proceed until a valid database is provided

User-facing output: The user is shown a clear warning that the database was not found and is re-prompted to enter a valid database name.

Case 6: SYSADMIN Without ACCOUNT_USAGE — Edge Case

Preconditions: SYSADMIN role active, warehouse set, no IMPORTED PRIVILEGES granted on the SNOWFLAKE database

Note: On a Standard Edition account, all roles inherently have ACCOUNT_USAGE. This case cannot be reproduced on a standard account, but is realistic on enterprise accounts where this default grant is revoked.

User Input: Run `USE ROLE SYSADMIN;` then run `governance health check`, provide valid database name when prompted

Expected Behavior: Partial report generated with reduced coverage

- **Phase 0 (access check):** ACCOUNT_USAGE is inaccessible; user is warned with exact remediation SQL
- **Execution:**
 - ACCOUNT_USAGE-dependent checks are skipped with Amber scoring
 - Independent checks (e.g., network policies) run where possible

User-facing output: The user is warned about missing access and provided with exact remediation SQL:

```
GRANT IMPORTED PRIVILEGES ON DATABASE SNOWFLAKE TO ROLE SYSADMIN;
```

The report clearly documents which checks were skipped and why, enabling the user to fix permissions and re-run.

Case 7: Warehouse Suspended Mid-Run — Failure

Preconditions: ACCOUNTADMIN role active, warehouse set with auto-resume disabled, all 8 sources accessible

User Input: Run `governance health check`. After Phase 0 completes, suspend the warehouse from a separate session:

```
ALTER WAREHOUSE COMPUTE_WH SET AUTO_RESUME = FALSE;  
ALTER WAREHOUSE COMPUTE_WH SUSPEND;
```

Expected Behavior: No report generated

- Phase 0 completes successfully — warehouse is active when the skill starts
- Phase 1 queries begin failing with "Warehouse is suspended"

- Skill detects persistent failure and stops execution

Case 8: SHOW NETWORK POLICIES Fails Due to Privilege — Failure

Preconditions: GOVERNANCE_ANALYST role active with IMPORTED PRIVILEGES on SNOWFLAKE database granting ACCOUNT_USAGE access, but no privilege to execute SHOW NETWORK POLICIES. Warehouse set. All other 7 sources accessible. Requires creating this custom role before testing.

Note: On a Standard Edition account, IMPORTED PRIVILEGES grants broad access, including SHOW NETWORK POLICIES. In enterprise environments with strict least-privilege policies, governance or compliance roles commonly have ACCOUNT_USAGE access without infrastructure-level SHOW command permissions. This makes it a realistic and important scenario for the target audience.

User Input: Run `USE ROLE GOVERNANCE_ANALYST`; then run `governance health check`, provide valid database name when prompted

Expected Behavior: Partial report generated with reduced coverage

- Phase 0 (source test): `SHOW NETWORK POLICIES` is inaccessible
- **Execution:**
 - Checks 11–12: Amber — skipped due to insufficient privileges
 - Check 10: Executes normally (uses `LOGIN_HISTORY`)
 - Other checks: Execute normally
- **Coverage:** Reported as “10 of 12 checks ran (83% coverage)”

User-facing output: The report indicates that network policy checks were skipped due to insufficient privileges and clearly documents which checks were affected and why.

How I Would Run These Evals

These tests would be run manually via Cortex Code CLI. Each case requires a specific session setup (`USE ROLE`, `USE WAREHOUSE`), then invoking the skill and verifying output matches expected behavior. The skill runs interactively — Phase 0 asks scoping questions via `ask_user_question` — so full automation is not practical without mocking that tool. For v1, manual verification is the right tradeoff.

Cadence: Run case 1 after any change to SKILL.md or sub-files. Run cases 1 and 2 before publishing a new version to other users. Run the relevant edge or failure case if you changed logic for that specific check. Run all 8 cases before publishing to a broader audience.

Signals to watch over time to catch regressions:

Critical regressions (skill runs but misleads):

- **Empty-state misclassification** — a check scoring Green when it should be Red on an empty result, the most dangerous regression because it gives false confidence
- **Score drift** — a check that previously scored Green now scores Amber or Red without any account changes, indicating SQL logic or ACCOUNT_USAGE data changed unexpectedly
- **False positives in identity checks** — Check 7 flagging service accounts as likely human or vice versa, indicating the `HAS_PASSWORD/HAS_RSA_PUBLIC_KEY` heuristic needs tuning

Operational regressions (skill fails visibly but does not mislead):

- **Increased skip rate** — more checks getting skipped across runs, suggesting Snowflake tightened permissions or the source-testing logic broke
- **Phase 0 failure rate** — if Phase 0 starts failing on accounts where it previously succeeded, a platform change may have altered permission requirements
- **PDF generation errors** — pandoc or weasyprint breaking after OS or Python updates, tracked by how often the fallback message appears instead of a PDF
- **Execution time creep** — if the skill takes noticeably longer, a query may be scanning more data than expected or hitting warehouse queuing
- **Edition-level feature availability** — classification and masking policy checks behave differently on Standard vs Enterprise Edition; any changes to Snowflake's edition feature matrix should trigger a review of checks 1, 2, 4, and 5

Observations

Case 1: Full Run, ACCOUNTADMIN, Baseline — Happy Path

Observed behavior largely matched expected behavior, with one exception. All 12 checks executed at 100% coverage. Checks 1 and 2 scored Red, Check 3 scored Amber, Checks 4 and 5 scored Amber, Checks 6-8 scored Amber, Check 9 scored Green, Checks 10-12 scored Amber. Data Protection domain scored Red, Identity and Access scored Amber, Auth and Network scored Amber. Overall score: Red. PDF generated successfully with colored RAG badges and scorecard.

One observation worth noting: Check 3 scored Amber rather than Red. A small number of tables already had tags applied from prior data infrastructure work, which was enough to push coverage above the Red threshold. This is expected behavior and demonstrates the skill correctly reads current account state rather than assuming a blank slate.

One surprise emerged during early skill development. Check 10 initially scored Red due to 13 failed logins exceeding the 11+ threshold. Pattern analysis showed all failures were from a single user at a single IP with benign error types — OAuth token expiry, MFA typos, and a missing second factor. There were no indicators of credential stuffing or brute force. The scoring logic in `auth_network.md` was updated to distinguish between attack patterns and operational noise. After the fix, Check 10 correctly scored Amber with contextual findings.

The findings from this run directly motivated the remediation steps taken before Case 2. Tags were applied to 80%+ of tables, and a restricted network policy was activated. Case 2 validated that all improvements were correctly reflected in the next run.

Case 2: Full Run, ACCOUNTADMIN, Post-Remediation — Happy Path

Observed behavior matched expected behavior. All 12 checks executed at 100% coverage. Checks 1 and 2 scored Red (Standard Edition — classification and masking unavailable). Check 3 improved from Amber to Green — 13 of 15 tables tagged (87%), exceeding the 80% threshold. Checks 4 and 5 scored Amber. Checks 6-10 and 12 scored the same as Case 1. Check 11 improved from Amber to Green — the account-level network policy was changed from `ALLOW_ALL_IPS` to `RESTRICTED_ACCESS`, limiting connections to a single known IP. Auth and Network domain improved from Amber to Green. Data Protection remained Red. Overall score: Red.

The key takeaway: the skill correctly detected both improvements — tag coverage and network policy — without any configuration changes to the skill itself. This validates the design as a repeatable audit tool: run it, remediate, run it again, and the scores reflect real progress.

One note on the overall score: the test design expects Amber on an Enterprise account where Data Protection improves to Green after classification and masking are applied. On the Standard Edition trial, Checks 1 and 2 remained Red due to feature unavailability rather than misconfiguration, keeping the overall score Red. On an Enterprise account with classification and masking configured, the overall score would improve to Amber, with Identity and Access as the only remaining gap due to the human user on powerful roles.

Case 5: Invalid Database Name in Phase 0 — Edge Case

Observed behavior matched expected behavior. When prompted for a database name, entering `FAKE_DB_12345` caused Phase 0 Step 5 to validate the input against `SHOW DATABASES`, detect the database did not exist, and re-prompt the user with a list of valid databases rather than proceeding to run checks.

One improvement over the expected behavior: the re-prompt was more helpful than specified — rather than simply warning that the database was not found, the skill listed all available user databases so the user could select a valid option immediately. This is a better UX than a bare error message and required no changes to the skill.

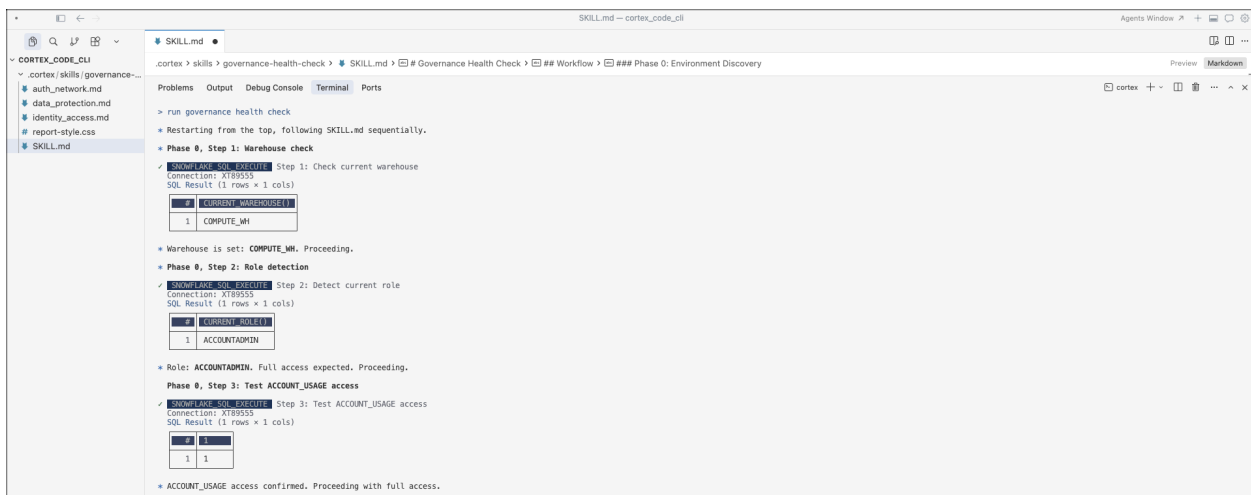
Case 7: Warehouse Suspended Mid-Run — Failure

Observed behavior did not match expected. When the warehouse was suspended mid-run, the skill detected the error but attempted to resume the warehouse by running `ALTER WAREHOUSE COMPUTE_WH RESUME` rather than retrying once and skipping as specified in the Error Handling section. This violates the read-only guardrail; the skill should never attempt to fix infrastructure state.

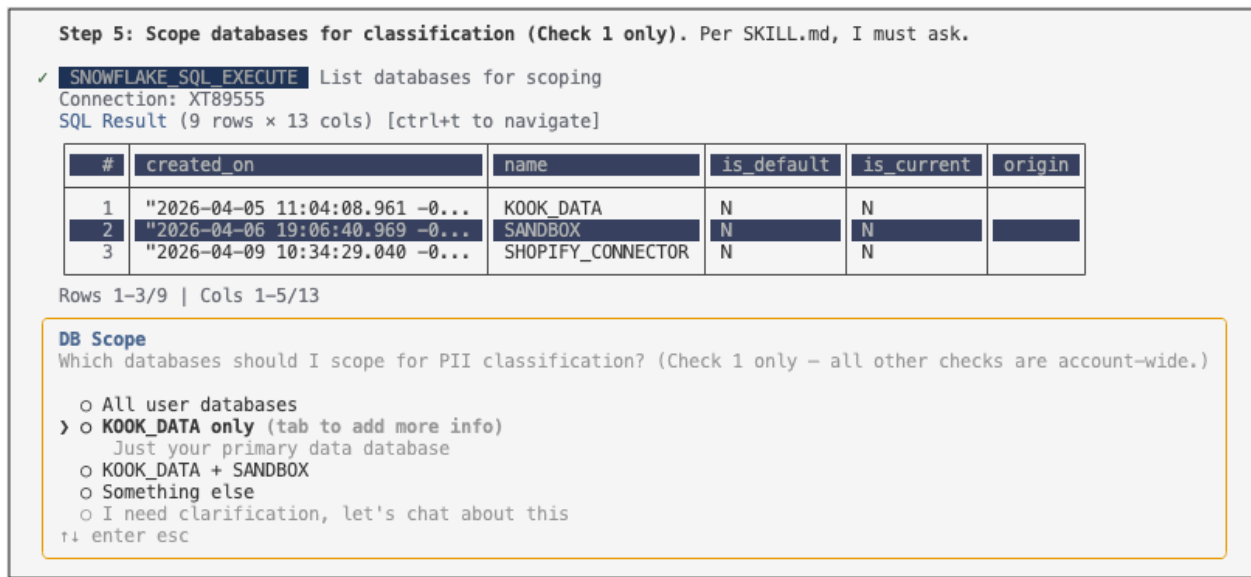
A fix would add an explicit instruction to the Error Handling section: "Do not attempt to fix infrastructure errors. Retry the query once, then skip."

Sample CLI Session

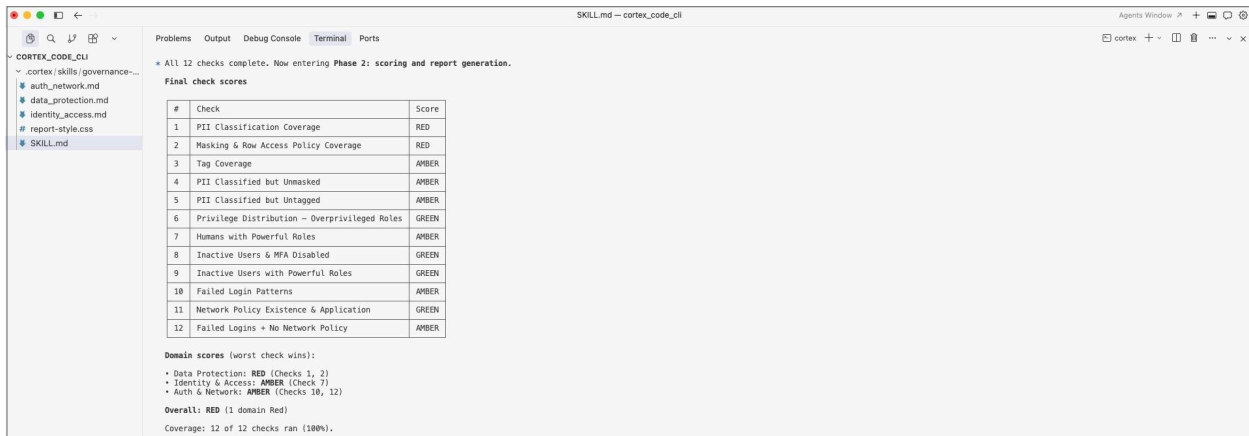
Skill invocation and Phase 0 environment discovery



Phase 0 interactive scoping: database selection



Phase 2 scoring summary: 12 checks with RAG scores and domain rollup



Resulting PDF (2 of 7 pages)

Data Governance Health Check Report

CONFIDENTIAL: This report contains security posture information. Do not share outside your security or governance team.

Generated: 2026-04-12 21:15 UTC
Role: ACCOUNTADMIN
Coverage: 12 of 12 checks ran (100%)
Data Protection scope: Check 1 scoped to KOOK_DATA. Checks 2-12 are account-wide.
Data sourced from ACCOUNT_USAGE views, which may have up to 3 hours of latency.
Small account detected — 1 user, 73 base tables. Thresholds calibrated for larger environments.

Overall Score: RED

This account has critical gaps in its data protection posture. No PII classification has ever been run and no masking or row access policies are applied anywhere, leaving sensitive data potentially exposed. Identity and access controls are partially in place — MFA is enabled and no inactive users exist, but a single human user holds both ACCOUNTADMIN and ORGADMIN. Auth and network controls show no suspicious activity: a network policy is applied at account level, and 13 failed login attempts in the last 30 days are all operational noise (OAuth token expiry and MFA typos from the account owner).

Top priorities:

- Run SYSTEM\$CLASSIFY on KOOK_DATA to identify PII columns, then create masking policies for any sensitive data found
- Create masking and row access policies — currently zero policies exist anywhere in the account

Check 7: Humans with Powerful Roles AMBER

Summary: 1 likely human user holds powerful admin roles

Findings: - JBORENSTEINLAWEE holds ACCOUNTADMIN — classified as “Likely Human” (has password, no RSA key) - JBORENSTEINLAWEE also holds ORGADMIN - This is the only user on the account

What this means: A single human user holds the two most powerful roles in Snowflake. While expected for a single-user account, this concentrates all administrative power without separation of duties.

Remediation:

— Consider creating a dedicated admin service account for automated tasks:

```
CREATE USER admin_service PASSWORD = '<strong_password>'
DEFAULT_ROLE = SYSADMIN
MUST_CHANGE_PASSWORD = FALSE;
```

— Or, for a single-user account, this may be acceptable.

— Review when adding more users to the account.

Check 8: Inactive Users & MFA Disabled GREEN

Summary: 0 inactive users, all users have MFA enabled, no active MFA bypasses

Findings: - No users inactive for 90+ days - All users have MFA enabled (HAS_MFA = TRUE) - No active BYPASS_MFA_UNTIL configurations

What this means: User hygiene is excellent — no stale accounts and MFA is universally enforced.

Remediation: No action needed.

Check 9: Inactive Users with Powerful Roles GREEN

Summary: 0 inactive users hold powerful roles

Findings: - No users with ACCOUNTADMIN, SECURITYADMIN, SYSADMIN, or ORGADMIN are inactive (90+ days)

What this means: No dormant accounts with elevated privileges exist, eliminating a common attack vector.

Remediation: No action needed.

Auth and Network AMBER

Check 10: Failed Login Patterns AMBER

Summary: 13 failed logins in last 30 days — high volume but benign, no credential stuffing or brute-force indicators detected

Governance Health Check — Confidential

Page 1 of 7

Governance Health Check — Confidential

Page 5 of 7